Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3320269.3384749

RESEARCH-ARTICLE

OptiSwap: Fast Optimistic Fair Exchange

LISA ECKEY, Technical University of Darmstadt, Darmstadt, Hessen, Germany

SEBASTIAN FAUST, Technical University of Darmstadt, Darmstadt, Hessen, Germany

BENJAMIN SCHLOSSER, Technical University of Darmstadt, Darmstadt, Hessen, Germany

Open Access Support provided by:

Technical University of Darmstadt

PDF Download
3320269.3384749.pdf
01 March 2026
Total Citations: 50
Total Downloads: 607

Published: 05 October 2020

Citation in BibTeX format

ASIA CCS '20: The 15th ACM Asia Conference on Computer and Communications Security
October 5 - 9, 2020
Taipei, Taiwan

Conference Sponsors:
SIGSAC

OptiSwap: Fast Optimistic Fair Exchange

Lisa Eckey
Technical University of Darmstadt
Germany
lisa.eckey@tu-darmstadt.de

Sebastian Faust
Technical University of Darmstadt
Germany
sebastian.faust@tu-darmstadt.de

Benjamin Schlosser
Technical University of Darmstadt
Germany
benjamin.schlosser@tu-darmstadt.de

ABSTRACT

Selling digital commodities securely over the Internet is a challenging task when Seller and Buyer do not trust each other. With the advent of cryptocurrencies, one prominent solution for digital exchange is to rely on a smart contract as a trusted arbiter that fairly resolves disputes when Seller and Buyer disagree. Such protocols have an *optimistic mode*, where the digital exchange between the parties can be completed with only minimal interaction with the smart contract. In this work we present OptiSwap, a new smart contract based fair exchange protocol that significantly improves the optimistic case of smart contract based fair exchange protocols. In particular, OptiSwap has almost no overhead in communication complexity, and improves on the computational overheads of the parties compared to prior solutions. An additional feature of OptiSwap is a protection mechanism against so-called grieving attacks, where an adversary attempts to violate the financial fairness of the protocol by forcing the honest party to pay fees. We analyze OptiSwap's security in the UC model and provide benchmark results over Ethereum.

CCS CONCEPTS

• **Security and privacy** → **Cryptography**.

KEYWORDS

fairness, cryptocurrency, smart contract, fair exchange, e-commerce, dispute resolution, optimistic, interactive

ACM Reference Format:
Lisa Eckey, Sebastian Faust, and Benjamin Schlosser. 2020. OptiSwap: Fast Optimistic Fair Exchange. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security (ASIA CCS '20), October 5–9, 2020, Taipei, Taiwan.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3320269.3384749

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASIA CCS '20, October 5–9, 2020, Taipei, Taiwan
© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-6750-9/20/10…$15.00
https://doi.org/10.1145/3320269.3384749

1 INTRODUCTION

Fair exchange of digital goods for money is a challenging task in the Internet when parties do not trust each other. Consider a two party protocol between a Seller $\mathcal{S}$ and a Buyer $\mathcal{B}$. On the one hand, $\mathcal{S}$ possesses some digital good x, which he is willing to sell for p coins. On the other hand, $\mathcal{B}$ wants to obtain x in exchange for the money. Since the Internet connects millions of users, it is reasonable to assume that both parties do not trust each other. While the Seller wants the guarantee that $\mathcal{B}$ gets to know x only if he pays p coins, the Buyer must be assured that he only needs to pay if $\mathcal{S}$ delivers the correct witness. It was shown by Pagnia and Gärtner in 1999 that such a fair exchange is not possible without a trusted third party (TTP) [18].

A common use case for a fair exchange protocol is the purchase of some digital file x, e.g., a movie, music file or software executable where only the hash $h = H(x)$ is known. The Buyer only wants to pay if the file x that he receives satisfies $h = H(x)$. At the same time, the Seller wants to be sure that if he delivers x which hashes to h, he gets his payment. The above can be generalized to a predicate function ϕ, where the Seller wants to sell a witness x that satisfies $\phi(x) = 1$. For instance, x may be some software, and ϕ defines test cases that the software has to satisfy. There exist different approaches to realize fair exchanges. One approach relies on a trusted middleman, which is often called escrow service, that implements the TTP [17]. A different approach is taken by protocols that rely on a blockchain to implement the TTP in case of dispute. Examples include the zero-knowledge contingent payment (ZKCP) protocols [4], and the FairSwap protocol [8]. In these works the TTP is implemented in a smart contract running on top of the blockchain. At a high-level these protocols work as follows. The Seller $\mathcal{S}$ encrypts the witness and sends it together with some auxiliary information to the Buyer. The Buyer checks the auxiliary information and, if accepted, it deposits money to the smart contract. Once the money is locked the Seller will reveal the secret key to the contract, which allows him to claim the money. Since the secret key now becomes publicly recorded on the blockchain, the Buyer can use it to decrypt the encrypted data.

Most relevant for our work is the protocol FairSwap [8]. FairSwap avoids expensive zero-knowledge proofs, and replaces it with a technique called *proof of misbehavior (PoM)*. A PoM allows the Buyer to punish the Seller in case the Seller sent an invalid witness x. This a-posterior approach of punishment has the advantage that FairSwap only needs to rely on simple hash function evaluations, thereby resulting

in a more efficient protocol for the users – in particular, it is more efficient if the witness x is large or the predicate function ϕ is complex. On the downside the communication between Seller and Buyer increases with the size of ϕ and x. Since the communication overhead occurs regardless of misbehavior, it is most likely the main bottleneck in practice.

We present an extension to the original FairSwap protocol of Dziembowski et al. [8] in order to improve the optimistic execution of the protocol, and thereby overcome one of the main bottlenecks of FairSwap in practice. This is achieved by incorporating an interactive dispute resolution sub-protocol, which removes the message overhead and thereby also improves the computational complexity of the two honest parties. Because our protocol is optimized for the *optimistic case* (which we believe is the standard case in practice, since parties can be punished when cheating), we call our protocol OptiSwap.

Another shortcoming of [8] is that grieving is not discussed. A corrupt Seller could force any Buyer to submit a large transaction (including the PoM) to the blockchain. This means a Seller can *grieve* the Buyer by forcing him to pay a lot of transaction fees. We will show how we can protect against grieving without relying on any security deposits when there are no disputes.

1.1 Contribution

We extend the FairSwap protocol by Dziembowski et al. [8] by making the dispute resolution interactive. Our contributions are summarized below:

- We present a challenge-response procedure for obtaining information about the evaluation of the predicate function ϕ. This procedure is only used in case of dispute, and allows to pinpoint a single step in the evaluation of ϕ where the Seller cheated. This approach significantly reduces the overheads of the FairSwap protocol from [8] in the optimistic case.
- In order to incentivise both parties to act honestly and prevent grieving, we incorporate fees into the dispute resolution sub-protocol.
- We implemented the OptiSwap protocol and improved it using the technique introduces by SmartJudge [20] in order to estimate the gas costs [1]. The obtained values are compared against the original FairSwap protocol.
- Finally, we provide comprehensive security evaluation of OptiSwap in the UC framework and show that it realizes a fair exchange functionality.

1.2 Related Work

It is well-known that fair exchange is impossible without a TTP [11, 18, 21]. In order to limit the role of a TTP [2, 5] proposed the distinction between the optimistic and pessimistic case of fair exchange. Since the emerge of blockchain technologies, smart contracts are considered as TTP, and there are numerous works that use smart contracts to improve on the fairness properties of cryptographic protocols [1, 3, 14–16]. Various works use smart contracts for fair exchange [4, 6, 8]. Below, we discuss related work that utilized interactive dispute resolution.

Interactive dispute resolution. Recent work incorporated an interactive dispute resolution handling for the case when two parties run into disagreement. The Arbitrum system was introduced by Kalodner et al. in 2018 [13]. They proposed a solution for the scalability issue related to many kinds of blockchain technologies. By performing heavy computation off-chain, the complexity of computation that is feasible to carry out backed-up by the blockchain is increased. Similar to OptiSwap in case of disagreement, Arbitrum runs an interactive dispute resolution protocol. This protocol narrows down the computation under dispute to a single instruction which is recomputed on-chain in order to resolve the whole disagreement. While OptiSwap also contains a challenge-response procedure as part of its dispute resolution protocol, we use a different challenge strategy. Concretely, in Arbitrum, computation is modeled using Turing machines resulting in a linear sequence of computation steps. For this type of data structure, a binary search algorithm is very efficient and, hence, it is used by Arbitrum. Within the OptiSwap protocol, we model computation using circuits that may have a far more complex topology than a linear list. Therefore, we make use of different challenge strategies.

A scalable verification solution for blockchain technologies called TrueBit was proposed by Teutsch and Reitwießner originally in 2017 [19]. It contains a dispute resolution layer which is executed in case of disagreement about the computation of some task. As in the Arbitrum system, TrueBit models computation using Turing machines. Hence, for resolving a dispute, the computation is narrowed down using a binary search procedure. As above the main difference to OptiSwap is that we use a circuit-based approach that allows more complex challenge strategies.

Optimistic mode. The SmartJudge architecture was proposed by Wagner et al. in 2019 [20]. It provides an abstraction of a two-party protocol that relies on the execution of a smart contract in case of dispute. Since our OptiSwap protocol is optimized for the optimistic mode, it is a perfect candidate to be combined with SmartJudge. SmartJudge splits a smart contract into a mediator and a verifier. Only the mediator must be executed in the optimistic mode and the verifier is only called in case of dispute. While SmartJudge provides an improvement for two-party protocol in the optimistic mode, it does not include a dispute resolution mechanism as this will depend on the actual protocol that instantiates the SmartJudge approach. We can combine our protocol with SmartJudge to further improve the optimistic mode and reduce on fee costs.

Fair exchange based on conditional dispute resolution. Concurrently to our work, Hall-Andersen presented an improvement of the FairSwap protocol called FastSwap [12]. FastSwap aims at an improved communication complexity in case of

[1] In Ethereum, parties have to pay for executed computation, where computation is measured in units of gas. Hence, the gas costs can be used as efficiency indicator.

both parties behaving honestly. This protocol incorporates an interactive challenge-response sub-protocol that is only executed in case of misbehavior. The work of Hall-Andersen focuses on modeling the verification computation using a computation model similar to Turing machines. This model results in a linear sequence of computation steps which allows a logarithmic number of rounds in the challenge-response phase. In contrast to FastSwap we use circuits for modeling computation. This allows us to use different challenge strategies depending on the topology of the verification circuit. Moreover, we explicitly add security fees into our protocol design to prevent grieving attacks. We stress that both protocols are compatible to each other which means that incorporated techniques can be adopted to be used in the other protocol as well. Since the FastSwap paper lacks a detailed evaluation of gas costs in the Ethereum system, a concrete comparison of both protocols is not possible. Finally, we provide a detailed security proof of our protocol in the universal composability framework in the full version of this paper [9]. While the FastSwap protocol is also modeled in this framework, a full security proof is missing.

2 PRELIMINARIES

In this section we introduce the main building blocks and describe the cryptographic primitives we need for designing OptiSwap. We denote integer variables by lower case letters and tuples by bold lower case letters (i.e., $\mathbf{a} = (a_1, \ldots, a_n)$) while sets are signaled by capital letters. The integer set $\{1, \ldots, n\}$ is denoted by $[n]$. For a probabilistic algorithm $\mathcal{A}$, $m' \leftarrow \mathcal{A}(m)$ denotes the output m' of $\mathcal{A}$ on input m. The symbol $\approx_c$ denotes the computational indistinguishability between two random variables. $s \xleftarrow{\$} S$ denotes that s is randomly sampled from the set S using a uniform distribution. Furthermore, we assume direct secure channels and synchronous communication happening in rounds. By using the notion of rounds, we abstract from the underlying process of consensus and confirmation times required when we interact with a blockchain system. The period of Δ rounds ensures that an honest party is able to wait for confirmation and to send a transaction afterwards.

Circuits. A key ingredient to our protocol will be a circuit which we use to model the predicate function that validates correctness of a witness. We will consider a witness $\mathbf{x}$ to be correct if the circuit evaluation of it returns 1, i.e. $\phi(\mathbf{x}) = 1$. Let X be a set of possible inputs and Γ an instruction alphabet. Each instruction $op : X^\ell \to X$ in Γ takes up to ℓ input values (fan-in: ℓ) and returns a single output value. A circuit ϕ over a value set X and an instruction set Γ is modeled as a directed acyclic graph (DAG) as follows. Its vertices $\phi_1, \ldots \phi_m$ are called *gates* and its edges represent the *wires* of ϕ and specify the information flow through the circuit. Any gate with in-degree 0 is called an *input gate*. The circuit contains n input gates, where $1 \leq n < m$, and each input gate ϕ_i outputs v_i to all outgoing edges. Every gate in ϕ with out-degree 0 is called *output gate*. We require that the circuit has a single output gate that evaluates to either 1 or 0. Every gate ϕ_i is labeled by a tuple $\phi_i = (i, op_i \in \Gamma, I_i \in [m]^\ell)$, evaluates the instruction op_i on the input values given by the circuit gates specified by the index set I_i, and outputs the result to all outgoing edges. There exists an ordering over all circuit gates such that for each gate ϕ_i the index i is greater than any index of a gate that provides an input to gate ϕ_i.

We note that the evaluation of a circuit is performed layer-by-layer starting from the input gates. Each gate evaluates its instruction on given inputs and propagates the result to the gates of the next layer. For simplicity we denote the outcome of the overall circuit as $\phi(\mathbf{x}) = \{0, 1\}$ and the result of the ith gate as $\phi_i(\mathbf{x}) = \mathsf{outp}_i$, for all $i \in [m]$. The depth δ of a circuit is the length of the longest path starting from any possible input gate up to its output gate.

Hash functions. We make use of cryptographic hash function $H : \{0,1\}^* \to \{0,1\}^\mu$, where $\mu \in \mathbb{N}$. These functions are used as building blocks for commitment and encryption schemes as well as Merkle trees. For our construction, we assume H to be a restricted programmable and observable random oracle. In the full version of this paper we give a detailed description of how we realize this primitive in the global random oracle model.

Commitment schemes. A commitment scheme allows to commit to a value m using the Commit-algorithm, which outputs a commitment c and an open value d. The commitment c hides m from others while the d can – at a later point – be used to reveal m. We require a commitment scheme to be *binding*, meaning that nobody can change a committed value m after c is known. Additionally, it must be *hiding*, i.e., nobody should learn the committed value from the public commitment c. Commitment schemes are formally defined in [10].

Symmetric encryption. We make use of a symmetric encryption scheme consisting of three ppt algorithms (KeyGen, Enc, Dec). The secret encryption key k is generated by the KeyGen-algorithm. The Enc-algorithm outputs a ciphertext z on input of the key k and a plaintext m. The Dec-algorithm returns the message m for given ciphertext c using key k. We assume a symmetric encryption scheme to be indistinguishable under chosen-plaintext attacks (IND-CPA) secure.

Merkle trees. Merkle trees are generated through the MTHash algorithm by iteratively hashing two hash values until only one hash – the root of the Merkle tree – remains. The root serves as a commitment on many chunks of data. The algorithm MTProof takes as input the tree and an index i and produces a logarithmically sized opening statement for the i-th chunk consisting of the neighbors on the path from the element to the root. The algorithm MTVerify is used to verify that the opening statement is correct in respect to the root. We refer to [8] for instantiation of these algorithms.

2.1 FairSwap

We briefly introduce the FairSwap protocol presented by Dziembowski et al. in 2018 [8] on a high level. The overall goal

of the protocol is a fair exchange of a digital good $\mathbf{x}$ against money. The Seller, who knows $\mathbf{x}$, initiates the protocol by sending an offer message to the Buyer. This message contains an encryption $\mathbf{z}$ of every step of the witness evaluation $\phi(\mathbf{x})$ under key k. By confirming this message the Buyer also transfers the payment of p coins to the judge contract. At this point the contract contains the money from the Buyer and a concise commitment on both $\mathbf{z}$ and ϕ. The Seller now reveals the key via the contract which allows the Buyer to decrypt the witness. If this witness is correct, the money will be transferred to the Seller (eventually). The more interesting case occurs when the witness that the Buyer received does not satisfy the agreed upon predicate, i.e., $\phi(\mathbf{x}) \neq 1$.

In this case, which we call the disagreement or pessimistic case, the Buyer can send a *proof of misbehavior (PoM)* to the contract. This proof is small, compared to both the witness and the predicate, but it is sufficient to indubitably prove that the received witness $\mathbf{x} = \mathsf{Dec}(k, \mathbf{z})$ does not satisfy the predicate, i.e. $\phi(\mathbf{x}) \neq 1$. If this can be shown to the judge contract, the coins will be refunded to the Buyer. Besides making sure that no coins can be stolen or locked, the protocol of [8] prevents cheating, namely that the Seller will get paid for a wrong witness or the Buyer can successfully complain about a correct one. Additionally, the designed PoM and all data stored inside the contract are kept very small to reduce the costs of the protocol.

Proof of Misbehavior (PoM).. In case of a dispute, Buyer and Seller argue about two different statements. The Seller wants the payment so he will claim that the transferred witness $\mathbf{x}$ is correct, i.e., $\phi(\mathbf{x}) = 1$ while the Buyer claims that it is incorrect $\phi(\mathbf{x}) = 0$. In order to figure out who is wrong the judge would need to re-evaluate the predicate $\phi(\mathbf{x})$. But since ϕ and $\mathbf{x}$ are potentially large, they should not be sent to the contract. Instead in [8] they consider a witness $\mathbf{x}$ that can be split in many elements $x_1, \ldots, x_n$ and a predicate ϕ which can be represented as a circuit with many gates $\phi_1, \ldots, \phi_m$. Now, instead of arguing about the evaluation of the overall predicate, it suffices to prove the dispute about the evaluation of a single gate ϕ_i where Buyer and Seller disagree on the gate output. Since the Buyer constructs and sends the PoM he has to prove that the Seller either sent the wrong witness or cheated during his computation of $\phi(\mathbf{x})$. This is why the Seller has to send the output for every evaluation of every gate of $\phi(\mathbf{x})$ to the Buyer and commit to all of these values (in the first message). These commitments allow the Buyer to prove that (i) either the output of $\phi(\mathbf{x}) = 0$ or (ii) there exists a gate $\phi_i \in \phi$ with inputs $\mathsf{in}_i^1, \ldots, \mathsf{in}_i^\ell$ where the Seller lied about the gate evaluation. The judge contract will then re-compute the gate in question and compare the outcome with the Seller's result, making it impossible to cheat successfully about the evaluation of ϕ_i. If the witness is wrong and the Seller computed the correct predicate, then the output of the last gate will reveal this fact, i.e. $\phi_m(\cdot) = 0$. But if the witness is correct, the Buyer will not be able to find a gate which is wrongly computed and the overall result will be 1, which means the judge will not accept any PoM.

A drawback of the FairSwap protocol is that the PoM is generated by the Buyer only considering information that the Seller sent in the first round. In particular the Seller needs to compute, store, encrypt and transfer the intermediary result of every single gate in the predicate evaluation $\phi(\mathbf{x})$. For predicate circuits with many gates, this can become a large overhead for the Seller and also for the Buyer, since in most cases when there is no dispute these information are not needed. We propose a different protocol, which does not require this overhead as long as Seller and Buyer agree. Only in case they run into a dispute, they can execute an interactive protocol which allows them to securely prove whether $\phi(\mathbf{x}) = 1$. Additionally, we will analyze how to fairly estimate transaction fees, which is not considered in the work of [8] at all.

3 OPTISWAP PROTOCOL DESCRIPTION

Similarly to [8] we consider an interactive protocol between a Buyer $\mathcal{B}$ who is willing to pay a Seller $\mathcal{S}$ p coins in exchange for a witness $\mathbf{x}$, if $\mathbf{x}$ satisfies a predicate function ϕ (i.e. $\phi(\mathbf{x}) = 1$). They use a smart contract as an adjudicator that stores the payment and either transfers the coins to $\mathcal{S}$ if the exchange is completed successfully or back to $\mathcal{B}$ if he does not receive his witness. The witness $\mathbf{x}$ can be split into n elements $x_1, \ldots, x_n$ and the predicate is represented as a circuit with m gates $\phi_1, \ldots, \phi_m$, a fan-in ℓ, a depth δ and a width ω as defined in Section 2.

3.1 OptiSwap Properties.

In order to ensure that neither the Buyer nor the Seller can cheat, we require the fair exchange protocol to have the following security properties:

- S1 Security against malicious Buyers: Honest Sellers will always receive their payment if the Buyer learns the witness $\mathbf{x}$ (cf. Sender fairness of [8]).
- S2 Security against malicious Sellers: Honest Buyers will either learn the correct witness $\mathbf{x}$ or will get their coins back (cf. Receiver fairness of [8]).
- S3 Security against grieving: In case of a dispute, the cheating party must always compensate the cheated party for any transaction fees paid during the dispute process (fee fairness).

Additionally to the security properties, we also need our protocol to be efficient. Therefore, we define the three following efficiency requirements:

- E1 Circuit independent communication: The size of all messages sent by Seller and Buyer in the optimistic case must be $\mathcal{O}(|\mathbf{x}|)$.
- E2 Constant round complexity in the honest case: If the correct witness is transferred without dispute, the protocol must run at most 5 rounds.
- E3 Upper bounded round complexity in the pessimistic case: Even in case of dispute, the protocol terminates

after finitely many rounds where the exact number of rounds is upper bounded by $\mathcal{O}(\min(\delta * \ell, \log(n) * \omega))$.[2]

In Section 5 we will argue why our protocol achieves all desired efficiency and security properties. A formal proof in the Universal Composability model can be found in the full version [9].

3.2 OptiSwap Protocol

At the core of the fair exchange protocol, Buyer and Seller use a smart contract, which has the authority over the payment and is used as a judge if the two parties disagree. Similarly to the FairSwap protocol of [8] the smart contract will evaluate a Proof of Misbehavior (PoM) in this case. This short proof is used to show that the Seller sent a wrong witness (cf. Section 2.1). In contrast to the FairSwap protocol, the Buyer does not already have the data to generate a PoM. Instead he needs to interactively challenge it from the Seller.

In the following we describe all steps on the protocol informally. A formal description for every party as well as a formal model for ledger and contracts can be found in the full version of this paper [9].

Round 1. The Seller starts the protocol by encrypting the witness $\mathbf{x}$ element by element, i.e., $\forall i \in [n] : z_i = \mathsf{Enc}(k, x_i)$. Then he commits to both, the encryption (by computing $r_z \leftarrow \mathsf{MTHash}(z_1, \ldots, z_n)$) and the used encryption key $(c, d) \leftarrow \mathsf{Commit}(k)$. Now, the judge contract can be initialized by the Seller. It is parameterized by the addresses of Seller and Buyer, the price p, public auxiliary information aux of the witness exchange, including fee and timing parameters (cf. Section 5). Additionally, both commitments r_z and c are stored in the contract, while the encrypted data $\mathbf{z}$ is sent to the Buyer directly.

Round 2. In the second round, the Buyer needs to confirm that he received the first message from the Seller and that he agrees with the parameters in the contract. In particular, he recomputes the Merkle tree root r_z of the ciphertext and checks all contract parameters including the price and auxiliary information. If he does not agree with any of them, he aborts the protocol. Otherwise, he sends a transaction to the judge contract which signals his acceptance and transfers the agreed upon payment to the contract. Note, that the Buyer cannot decrypt the witness at this point, because he does not know the key k and neither the encryption $\mathbf{z}$ nor the commitment c reveal any information about $\mathbf{x}$ or k.

Round 3. Now that the coins are locked in the contract, the Seller can reveal the encryption key. In order to make this action publicly verifiable he stores the key in the smart contract, which verifies its correctness ($\mathsf{Open}(c, k, d) = \mathsf{true}$).

Round 4. Since the key is stored publicly, the Buyer can now decrypt the witness: $\forall z_i \in \mathbf{z} : \mathsf{Dec}(k, z_i) = x_i$. At this point we distinguish two cases. In the optimistic case the witness was correct, which the Buyer verifies by running $\phi(\mathbf{x}) = 1$. In this case he confirms the successful transfer to the contract, which triggers the payment to the Seller. Even when the Buyer does not send this message, the Seller can collect the coins after some timeout has passed in the next round. This timeout is necessary for the second case which we call the dispute or pessimistic case, in which the Buyer finds that the received witness is incorrect, i.e., $\phi(\mathbf{x}) = 0$. In this case the Buyer will start the interactive dispute protocol, which will effectively freeze the coins in the contract until a malicious party is found.

[2]There are different dispute resolution procedures which are more or less efficient depending on the circuit parameters. The procedures are described in Section 3.5. In particular, the round complexity for the file sale application is $\mathcal{O}(\log(n))$.

3.3 Interactive Dispute Handling

The dispute protocol is based on an interactive challenge-response procedure between Seller $\mathcal{S}$ and Buyer $\mathcal{B}$. The goal is to identify a single gate within the circuit in which the evaluations of both parties differ. Then the Buyer can use this information to prove that he received a false witness using a Proof of Misbehavior (PoM). Recall that for evaluating the PoM about the i-th gate, the judge requires as input the gate ϕ_i, its inputs $\mathsf{in}_i^1, \ldots, \mathsf{in}_i^\ell$ and the gate output computed by $\mathcal{S}$: outp_i. The Buyer will query these information from $\mathcal{S}$ by repeatedly asking for outputs of circuit gates. All queries and responses will be recorded by the judge contract.

The Buyer starts by querying the result for the last gate ϕ_m, which is the overall output of the evaluation of $\phi(\mathbf{x})$. If the Seller replies with $\mathsf{outp}_m = 0$ $\mathcal{B}$ can immediately prove that $\mathcal{S}$ misbehaved. If the Seller cheats, he could claim that $\mathsf{outp}_m = 1$, which means both parties have different results on the output of the gate. Now, $\mathcal{B}$ challenges the outputs of all input gates of ϕ_m and obtains $\mathsf{in}_m = (\mathsf{in}_m^1, \ldots, \mathsf{in}_m^\ell)$ which he can compare to his own computed values. Again the Seller can either send the values which correctly evaluated to his alleged result ($\phi_m(\mathsf{in}_m) = \mathsf{outp}_m$), or he could send mismatched values, such that $\phi_m(\mathsf{in}_m) \neq \mathsf{outp}_m$. If the second case happens, again the Buyer triggers the PoM evaluation of the contract, which will verify that the Seller is cheating.

During this dispute resolution, the contract verifies the alternating participation of Seller and Buyer. In every repetition j, the Buyer challenges the input gates in_j to one or more of the challenged gate of the previous round $\mathsf{outp}_j \in \mathsf{in}_{(j-1)}$. The Seller has to respond by sending all corresponding output values to the challenged gates. This way the Buyer can evaluate $\phi_i(\mathsf{in}_j) = \mathsf{outp}_j$. Seller and Buyer repeat this challenge-response procedure until one of the following cases happens:

(i) The Seller does not respond to the challenge (in time). In this case the Buyer can request the money back from the contract.
(ii) The Buyer does not send a new challenge or a PoM (in time). In this case the Seller can request the payment.
(iii) The Seller responds incorrectly to a challenge, i.e., $\phi_i(\mathsf{in}_j) \neq \mathsf{outp}_j$. In this case the Buyer reveals the instruction of the gate ϕ_i and proves with a Merkle tree proof that this gate is part of the circuit ϕ. Now the judge contract can verify that the Seller misbehaved and sends the coins back to the Buyer.

(iv) The Seller sends a wrong witness. This case can happen, when one of the requested inputs is an element of the witness but the Seller responds with a different element x_i than the Buyer decrypted x_i'. In this case the Buyer will send the ciphertext value z_i to the contract and includes a Merkle tree proof that z_i is the i-th element of $\mathbf{z}$ (using r_z). Again the contract will be able to verify this cheating and reward the coins back to the Buyer.

(v) The last case occurs when the Buyer exhausts allowed challenge limit a_ϕ. This means the Buyer could not prove cheating of the Seller in time and the contract will send the payment to the Seller. The challenge limit a_ϕ is fixed for every circuit before the protocol start and denotes the maximum number of gates that the Buyer needs to challenge in order to prove cheating.

Challenge limit parameter. As mentioned before, the judge is responsible for limiting the number of possible challenges. To this end, the judge is initialized with a *challenge limit* parameter a_ϕ during the first protocol round as part of the auxiliary information aux. This parameter defines the maximal number of challenged circuit gates and it is important that both parties agree on it. The value must be large enough such that the Buyer is able to prove misbehavior for any possible gate or input value. At the same time it should be as small as possible to reduce the runtime and cost of a dispute.

3.4 Formal Description

In this section, we give a formal definition of the judge smart contract and the two protocol parties named Seller and Buyer. We formally prove the security of our construction within the universal composability framework in the full version of this paper [9] .

In order to shorten the formal description of the judge smart contract and the protocol parties, parts of the computations are extracted to small algorithms. We denote these algorithms by expressive names such that the reader is able to follow our description. See the full version of this paper for a detailed description of them.

We start the formal protocol description with the definition of the judge given below. It models a smart contract that interacts with both protocol parties, $\mathcal{S}$ and $\mathcal{B}$, as well as the global ledger $\mathcal{L}$ and a global random oracle $\mathcal{H}$. We present details about $\mathcal{L}$ and $\mathcal{H}$ in the full version of this paper. In comparison to the judge functionality used by FairSwap, $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$ as defined below additionally contains functions for handling the challenge-response procedure as well as timeouts.

Judge hybrid functionality $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$

The judge stores two addresses $pk_{\mathcal{S}}$ and $pk_{\mathcal{B}}$, a commitment c, a decryption key k, Merkle tree root hashes r_z, r_e, and r_ϕ, the maximum number of challenged gates $a_\phi \in \mathbb{N}$, a price $p \in \mathbb{N}$, and the fee parameters $f_{\mathcal{S}}, f_{\mathcal{B}} \in \mathbb{N}$. Moreover, it contains a state s which is initially set to $s = \mathit{start}$ and it stores the most recent challenge query Q_r and the most recent response R_r. Additionally, $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$ contains two timeout parameters $T_1, T_2 \in \mathbb{N}$ and corresponding variables $t_1, t_2 \in \mathbb{N}$, respectively.

All stored values depend on the session identifier id corresponding to one protocol execution. Hence, the values are functions evaluating on the session identifier, e.g., the state s on input id is initially set to $s(id) = \mathit{start}$. For the sake of simplicity, the input parameter id is omitted in the following description.

Initialization

On receiving $(\mathit{initialize}, id, c, r_z, r_e, r_\phi, a_\phi, p, f_{\mathcal{S}}, f_{\mathcal{B}})$ from $\mathcal{S}$ when $s = \mathit{start}$, store $pk_{\mathcal{S}}, c, r_z, r_e, r_\phi, a_\phi, p, f_{\mathcal{S}}$, and $f_{\mathcal{B}}$. Output $(\mathit{active}, id, c, r_z, r_e, r_\phi, a_\phi, p, f_{\mathcal{S}}, f_{\mathcal{B}})$ and set $s = \mathit{active}$.

On receiving (accept, id) from $\mathcal{B}$ when $s = \mathit{active}$, store $pk_{\mathcal{B}}$ and send $(\mathit{freeze}, id, \mathcal{B}, p)$ to $\mathcal{L}$. If the response is $(\mathit{frozen}, id, \mathcal{B}, p)$, set $s = \mathit{initialized}$ and output $(\mathit{initialized}, id)$.

Abort

On receiving (abort, id) from $\mathcal{S}$ when $s = \mathit{active}$, set $s = \mathit{finalized}$, and output $(\mathit{aborted}, id)$.

On receiving (abort, id) from $\mathcal{B}$ when $s = \mathit{initialized}$, send $(\mathit{unfreeze}, id, \mathcal{B})$ to $\mathcal{L}$, set $s = \mathit{finalized}$, and output $(\mathit{aborted}, id)$.

Revealing

On receiving $(\mathit{reveal}, id, k, d)$ from $\mathcal{S}$ when $s = \mathit{initialized}$ and $\mathsf{Open}(c, k, d) = \mathsf{true}$, output $(\mathit{revealed}, id, k, d)$, set $t_1 = \mathsf{now}$, and $s = \mathit{revealed}$.

Challenge-Response

On receiving $(\mathit{challenge}, id, Q)$ from $\mathcal{B}$ when $s = \mathit{revealed}$ or $s = \mathit{responded}$, $|Q| \leq a_\phi$, and $\forall i \in Q : 1 \leq i \leq m$ send $(\mathit{freeze}, id, \mathcal{B}, f_{\mathcal{B}} * |Q|)$ to $\mathcal{L}$. If it responds with $(\mathit{frozen}, id, \mathcal{B}, f_{\mathcal{B}} * |Q|)$, set $Q_r = Q$, $a_\phi = a_\phi - |Q|$, $t_2 = \mathsf{now}$, output $(\mathit{challenged}, id, Q)$ and set $s = \mathit{challenged}$.

On receiving $(\mathit{respond}, id, R_q)$ from $\mathcal{B}$ when $s = \mathit{challenged}$, send $(\mathit{freeze}, id, \mathcal{S}, f_{\mathcal{S}} * |Q_r|)$ to $\mathcal{L}$. If it responds with $(\mathit{frozen}, id, \mathcal{S}, f_{\mathcal{S}} * |Q_r|)$, set $R_r = R_q$, $t_2 = \mathsf{now}$, output $(\mathit{responded}, id, R_q)$ and set $s = \mathit{responded}$.

Timeouts

On receiving $(challenge\ timeout, id)$ from $\mathcal{S}$ when $s = responded$ and $t_2 + T_2 \leq \mathsf{now}$, set $s = finalized$, send $(unfreeze, id, \mathcal{S})$ to $\mathcal{L}$, and send $(sold, id)$ to $\mathcal{S}$.

On receiving $(response\ timeout, id)$ from $\mathcal{S}$ when $s = challenged$ and $t_2 + T_2 \leq \mathsf{now}$, set $s = finalized$, send $(unfreeze, id, \mathcal{B})$ to $\mathcal{L}$, and send $(cancelled, id)$ to $\mathcal{B}$.

Finalizing

On receiving $(complain, id)$ from $\mathcal{B}$ when $s = responded$, set $s = finalized$ and do one of the following:
- If $\mathsf{ValidateResponse}(Q_r, R_r, r_e) = \mathsf{false}$ send $(unfreeze, id, \mathcal{B})$ to $\mathcal{L}$, send $(valid, id)$ to $\mathcal{B}$ and $(not\ sold, id)$ to $\mathcal{S}$.
- Otherwise, send $(unfreeze, id, \mathcal{S})$ to $\mathcal{L}$, send $(invalid, id)$ to $\mathcal{B}$ and $(sold, id)$ to $\mathcal{S}$.

On receiving $(prove, id, \pi)$ from $\mathcal{B}$ when $s = responded$, set $s = finalized$ and do one of the following:
- If $\mathsf{Judge}(k, r_z, r_e, r_\phi, \pi) = 1$ send $(unfreeze, id, \mathcal{B})$ to $\mathcal{L}$, send $(valid, id)$ to $\mathcal{B}$ and $(not\ sold, id)$ to $\mathcal{S}$.
- Otherwise, send $(unfreeze, id, \mathcal{S})$ to $\mathcal{L}$, send $(invalid, id)$ to $\mathcal{B}$ and $(sold, id)$ to $\mathcal{S}$.

On receiving $(finalize, id)$ from $\mathcal{B}$ when $s = revealed$, send $(unfreeze, id, \mathcal{S})$ to $\mathcal{L}$, set $s = finalized$ and send $(sold, id)$ to $\mathcal{S}$.

On receiving $(finalize, id)$ from $\mathcal{B}$ when $s = revealed$ and $t_1 + T_1 \leq \mathsf{now}$, send $(unfreeze, id, \mathcal{S})$ to $\mathcal{L}$, set $s = finalized$, and send $(sold, id)$ to $\mathcal{S}$.

We continue with the definition of an honest Seller and Buyer in the OptiSwap protocol. Both protocol parties are formally defined below.

Honest Seller and Buyer Description

The protocol description comprises a definition of the behavior of the honest Seller $\mathcal{S}$ and Buyer $\mathcal{B}$. Within the protocol, two timeouts $T_1, T_2 \in \mathbb{N}$ are used. These timeouts are defined and handled by the judge smart contract functionality $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$.

Initialization

$\mathcal{S}$: On receiving $(sell, id, \mathbf{x}, \phi, p, f_\mathcal{S}, f_\mathcal{B})$, where $\mathbf{x} = (x_1, \ldots, x_n) \in (\{0,1\}^\lambda)^n$, ϕ being a circuit with challenge limit property a_ϕ, $\phi(\mathbf{x}) = 1$, and $p, f_\mathcal{S}, f_\mathcal{B} \in \mathbb{N}$, $\mathcal{S}$ samples a key $k \leftarrow \mathsf{Gen}(1^\kappa)$, computes a commitment $(c, d) \leftarrow \mathsf{Commit}(k)$, and executes the presetup algorithm $(\mathbf{z}, r_z, r_e, r_\phi) \leftarrow \mathsf{Presetup}(\phi, \mathbf{x}, k)$. He sends $(initialize, id, c, r_z, r_e, r_\phi, a_\phi, p, f_\mathcal{S}, f_\mathcal{B})$ to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$ and $(sell, id, \mathbf{z}, \phi)$ to $\mathcal{B}$. He continues in the *Revealing*-phase.

$\mathcal{B}$: On receiving (buy, id, ϕ), where ϕ being a circuit with challenge limit property a_ϕ, $\mathcal{B}$ checks if he received a message $(sell, id, \mathbf{z}, \phi)$ from $\mathcal{S}$ beforehand. Then, he computes $r_z = \mathsf{MTHash}(\mathbf{z})$ and $r_\phi = \mathsf{MTHash}Mtree(\phi)$. Upon receiving $(active, id, c, r_z, r_e, r_\phi, a_\phi, p, f_\mathcal{S}, f_\mathcal{B})$ from $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$, $\mathcal{B}$ responds with $(accept, id)$ to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$ and continues in the *Revealing*-phase.

Revealing

$\mathcal{S}$: On receiving $(initialized, id)$ from $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$, $\mathcal{S}$ reveals his key generated in the *Initialization*-phase by sending $(reveal, id, k, d)$ to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$. He continues in the *Finish*-phase.
On receiving $(abort, id)$, $\mathcal{S}$ sends $(abort, id)$ to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$ and continues in the *Finish*-phase.

$\mathcal{B}$: On receiving $(revealed, id, k, d)$ from $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$, $\mathcal{B}$ decrypts the witness $\mathbf{x}' = \mathsf{Dec}(k, \mathbf{z})$, outputs $(revealed, id, \mathbf{x}')$, and evaluates $\phi(\mathbf{x}')$. If the output is 1, he sends a $(finalize, id)$ message to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$, outputs $(bought, id, \mathbf{x}')$, and terminates the protocol. Otherwise, he sets the set of obtained responses $R := \emptyset$, creates a challenge query by executing $(Q, H') \leftarrow \mathsf{NextChallenge}(\phi, R, H)$, updates the most recent query $Q_r = Q$, and sends $(challenge, id, Q)$ to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$. $\mathcal{B}$ continues in the *Challenge-Response*-phase.
On receiving $(abort, id)$, $\mathcal{B}$ sends $(abort, id)$ to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$ and continues in the *Finish*-phase.

Challenge-Response

$\mathcal{B}$: On receiving $(responded, id, R_q)$ from $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$, $\mathcal{B}$ computes $check = \mathsf{ValidateResponse}(Q_r, R_q, r_e)$. If $check = \mathsf{false}$, he sends $(complain, id)$ to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$ and continues in the *Finish*-Phase. Otherwise, $\mathcal{B}$ adds the received responses to the set of all responses $R = R \cup R_q$ and computes $\pi = \mathsf{GenerateProof}(k, \phi, R)$. If $\pi \neq \mathsf{false}$, he sends $(prove, id, \pi)$ to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$ and continues in the *Finish*-phase. Otherwise if $\pi = \mathsf{false}$, $\mathcal{B}$ generates the next challenge

$(Q, H') \leftarrow \mathsf{NextChallenge}(\phi, R, H)$ and sends $(challenge, id, Q)$ to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$.
If no $(responded, id, \cdot)$ message was received after timeout T_2, $\mathcal{B}$ sends a $(response\ timeout, id)$ message to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$ and continues in the *Finish*-phase.

$\mathcal{S}$: On receiving a $(challenged, id, Q)$ message from $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$, he generates the corresponding response by computing $R_q \leftarrow \mathsf{GenerateResponse}(\phi, \mathbf{x}, k, Q)$ and sends $(respond, id, R_q)$ to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$.
If no $(challenged, id, \cdot)$ message was received after timeout T_2, $\mathcal{S}$ sends a $(challenge\ timeout, id)$ message to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$ and continues in the *Finish*-phase.

Finish

$\mathcal{S}$: On receiving $(sold, id)$, $(not\ sold, id)$, or $(aborted, id)$ from $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$, $\mathcal{S}$ outputs this message and terminates the protocol.
On receiving a $(challenged, id, Q)$ message from $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$, he generates the corresponding response by computing $R_q \leftarrow \mathsf{GenerateResponse}(\phi, \mathbf{x}, k, Q)$ and sends $(respond, id, R_q)$ to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$. He continues in the *Challenge-Response*-phase.
If no $(sold, id)$ message and no $(challenged, id, \cdot)$ message from $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$ was received after timeout T_1, he sends a $(finalize, id)$ message to $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$.

$\mathcal{B}$: On receiving $(invalid, id)$ from $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$, $\mathcal{B}$ outputs $(bought, id, \mathbf{x}')$ and terminates the protocol.
On receiving $(valid, id)$ or $(cancelled, id)$ from $\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}$, $\mathcal{B}$ outputs $(not\ bought, id, \mathbf{x}')$ and terminates the protocol.
On receiving $(aborted, id)$, $\mathcal{B}$ outputs this message and terminates the protocol.

3.5 Extensions and Optimizations

The challenge-response procedure is very expensive to run compared with the optimistic case execution. We distinguish three different dimensions that influence the cost for running the dispute: (1) The large *data storage* which consists of all values the Seller submits during dispute increases the transaction costs drastically. (2) The runtime of the sub-protocol mainly depends on the *number of challenge rounds* which increases with the depth of the circuit ϕ. (3) Additionally, the *complexity of the judge contract* also depends on ϕ. The main factor here is the size of the instruction set of the circuit, since the judge needs to be able to evaluate any possible gate. Depending on the circuit and the application, it can be smart to optimize the dispute procedure.

Reducing contract storage. Storing data in a contract is costly since it becomes part of the blockchain state and every single miner has to allocate space for this data. Therefore, the costs of running a contract can be reduced if storage is re-used. In the case of our judge contract, the storage requirements of the dispute sub-procedure can be optimized. Instead of storing all challenges and responses, it only stores the latest ones. This means the Seller needs to commit to all computed values using a Merkle tree root r_e computed over the intermediate values of the evaluation $\phi(x)$. Now, for every revealed element the Seller includes a Merkle proof about the item, which is verified and stored by the Buyer. This data allows him to generate a proof including elements which have been revealed in a previous round but are no longer stored in the contract.

Reducing round complexity. The worst case round complexity is fixed for every circuit. In order to reduce the maximal and average number of challenge rounds the parties could agree to challenge and reveal outputs of multiple gates at the same time. Depending on the circuit they could also agree on different challenge strategies which can again help to reduce this parameter. For circuits that are very balanced and have a tree like structure it makes a lot of sense to start with the last gate and always query one predecessor gate, for which the Seller provided a divergent output. This basic strategy results in an upper bound for the overall round complexity in case of dispute of $4 + 2 * ((\delta - 1)\ell + 1)$ rounds. For very deep circuits with a low width ω this strategy is not optimal but instead a binary search will be better suited (cf. Figure 1). This strategy results in an upper bound of $4 + 2 * (\lceil \log_2 \delta \rceil * \omega + 1)$ rounds. Knowing the circuit-dependent parameters δ, ℓ and ω, the more efficient strategy can be chosen. Both strategies can even be combined for optimal results. Another option is to combine multiple gates into one by defining new combined instructions. This can change the overall structure of the circuit and lower its challenge costs, creating trade-off between round complexity and contract complexity.

Reducing contract complexity. For complicated circuits with many instructions the logic of the judge contract can get very complex. Since the majority of its logic is only required for dispute handling and evaluating the PoM, the contract can be split into two parts. This is an approach which has been introduced in [20]. It allows to deploy the dispute logic only in case the Buyer triggers the dispute. This trick drastically reduces the deployment costs and, hence, lowers the costs of the optimistic case.

To generalize over all improvements made in this section we will abstract from specific challenge strategies and encapsulate this into a function $\mathsf{NextChallenge}$ that selects the next challenge query based on the circuit and the previous response (more on this in the full version of this paper [9]).

At this point we have presented most features of OptiSwap and can give a full list of all required auxiliary information $\mathsf{aux} = (c, r_z, r_e, r_\phi, a_\phi, f_\mathcal{S}, f_\mathcal{B}, T_1, T_2)$. The parameter r_ϕ denotes a commitment to the verification circuit ϕ and $f_\mathcal{S}, f_\mathcal{B}$ being the fee parameter, which will be explained in the next

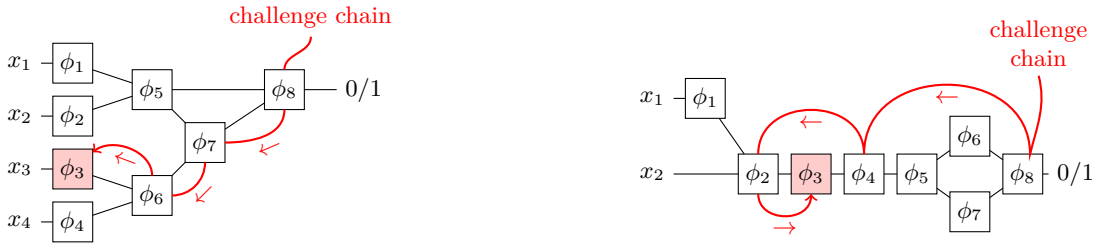


Figure 1: Back-to-front (left) and binary search (right) challenge strategies.

section. Moreover, aux contains two timeout parameters T_1 and T_2.

4 OPTISWAP EVALUATION AND TRANSACTION FEES

In this section we evaluate the efficiency of our protocol. We start with a discussion about the communication complexity in the honest execution, which is the main improvement of OptiSwap compared to FairSwap. Then, we analyze the runtime and costs of our protocol. We also discuss how we incorporated transaction fees into OptiSwap to prevent grieving. In order to illustrate the evaluation of our protocol, we consider the file sale application which we describe in the following.

File sale application. For exact measurements and better comparison we implemented[3] the file sale application of [8] as a concrete instantiation of circuit ϕ. In this case the witness is a file $\mathbf{x}$ consisting of n chunks $x_1, \dots x_n$ of size $\lambda = 512$ Byte and it is identified via its Merkle hash $h = \mathsf{MTHash}(\mathbf{x})$. The verification circuit ϕ computes the Merkle hash of the input and compares it with the expected value, i.e., $\phi(\mathbf{x}) = 1 \Leftrightarrow \mathsf{MTHash}(\mathbf{x}) = h$. The required instruction set consists of a Hash function evaluation and one check-if-equal instruction.

4.1 Communication Complexity in the Optimistic Execution

The goal of OptiSwap is to improve the optimistic execution. Assuming both Seller and Buyer behave honestly, the transferred witness $\mathbf{x}$ is correct and unlike in FairSwap there is no necessity to exchange information about the predicate evaluation $\phi(\mathbf{x})$ in order to prove misbehavior. Concretely, in the optimistic execution of OptiSwap, most of the protocol messages have a constant size (independent of the witness). Only in the first round, the data transferred from Seller to Buyer depends on the size of the witness $\mathbf{x}$. This in particular means that compared to previous solutions like FairSwap [8], the size of this message is independent of the circuit ϕ. Thus, OptiSwap removes the overhead of FairSwap from the first protocol message, where this overhead is given as the ratio of the message size $|\mathbf{z}|$ to the size of the witness $|\mathbf{x}|$. To show the advantage of our OptiSwap protocol compared to FairSwap, we consider three different use cases and show the resulting communication overhead in FairSwap. We start by taking a look at the overhead in the file sale application. Since our protocol can also be used for arbitrary applications that are based on an arithmetic or a Boolean verification circuit, we then analyze the communication overhead for matrix multiplication as well as an AES-256 bug bounty application. Table 1 lists the communication overhead of the first protocol message in FairSwap for the different applications. We stress that the overhead as defined above in OptiSwap is always 1, which basically means there is no overhead.

First, we consider the file sale application as described above. For the file sale application it holds that the size of the verification circuit depends on the size of a single file chunk. The smaller the file chunks, the larger the circuit size and the more data has to be transferred in the first protocol message in FairSwap. Table 1 shows that the overhead increases for the file sale application if the size of each file chunk decreases. This is a drawback since a small file chunk size is preferable over a big one as we will see in the following. In case of dispute, the Buyer is responsible for creating a PoM. Assuming the Seller sent an incorrect witness, the PoM might contain a small part of the file in order to show that its hash results in a different hash value than computed by the Seller. Since the PoM is sent to the judge smart contract, the data is on-chain and, hence, publicly visible. This data leakage increases in the size of the file chunks. An additional drawback is that more storage on the Blockchain results into higher gas costs. In contrast, the smaller the file chunk size the less data needs to be published in the worst case. Using a file chunk size of 1 Byte, the size of the encoding in the FairSwap protocol is 33 times the size of the witness.

It is easy to see that this overhead is way too high for large witnesses, e.g., 1 GByte files, therefore, it is unreasonable to use small file chunks in the FairSwap protocol. On the contrary, OptiSwap does not suffer from the overhead and, hence, the file chunk size can be chosen arbitrarily small. It is important to note that a small file chunk size also results in a higher circuit depth leading to an increased number of rounds in the dispute resolution. However, halving the file chunk size increases the circuit depth only by one.

Second, we consider matrix multiplication as an example for arithmetic circuits. In this scenario, the Buyer asks for two quadratic matrices whose product is equal to a predefined result. We assume that each element in one of the two

[3] github.com/CryBtoS/OptiSwap

Table 1: Communication overhead in FairSwap [8] for different applications.

Application	Witness size $\lvert\mathbf{x}\rvert$	Encoding size $\lvert\mathbf{z}\rvert$ in [8]	Communication overhead $\frac{\lvert\mathbf{z}\rvert}{\lvert\mathbf{x}\rvert}$ in [8]
file sale (32 Bytes chunk size)	1 GByte	2 GBytes	2
file sale (4 Bytes chunk size)	1 GByte	9 GBytes	9
matrix multiplication (2×2)	256 Bytes	864 Bytes	3.38
matrix multiplication (10×10)	6 400 Bytes	73 568 Bytes	11.5
AES-256	32 Bytes	1 088 000 Bytes	34 000

Table 2: Gas cost for the file sale application in OptiSwap.

Standalone		
deploy	2 273 398 gas	$ 5.94
optimistic case	101 307 gas	$ 0.26
pessimistic case	6 412 569 gas	$ 16.77

Split Contract		
optimistic deploy	952 939 gas	$ 2.49
pessimistic deploy	1 962 992 gas	$ 5.13

input matrices is represented by 32 Bytes[4]. The resulting communication overhead increases with the parameter n for $n \times n$-matrix multiplication. Table 1 lists the concrete overhead for $n = 2$ and $n = 10$. If we use larger values for n, the overhead will be even more significant.

As a third use case for evaluating the overheads of our solution compared to FairSwap we consider a bug bounty scenario. Here, the Buyer offers a bounty for a Seller if Seller can break AES-256 by providing an input to the encryption algorithm such that the output is equal to a predefined ciphertext. We use a Boolean circuit of the AES-256 cipher containing 34 000 gates for the analysis. Since the circuit size is high compared to the input size, the overhead becomes very huge. Instead of just transferring the 32 Byte input, the encoding size is greater than 1 MByte. Although the output of a gate within a Boolean circuit is just one bit, we consider the encryption scheme as proposed in [8] where each ciphertext has a size of 32 Byte. This can possibly be improved using a different encryption scheme that can efficiently be realized on Ethereum.

We summarize that our OptiSwap protocol is always preferable over the FairSwap protocol in case of two honest parties. The advantage increases for applications with a small witness size and a high verification circuit size. Next, we analyze the gas costs of our reference implementation of the file sale application. We show that our protocol is affordable and costs are roughly the same as for the FairSwap protocol.

4.2 Runtime and Gas Costs

In Ethereum, transaction fees are paid in *gas* and every instruction of the Ethereum virtual machine code has a gas value assigned to it. This gives a deterministic gas amount for every transaction. The total amount of gas required for a transaction can be used as efficiency indicator. The gasprice[5] can be set individually for every transaction and describes the exchange rate between Ether and gas. We assume that the gasprice will be fixed for the duration of the protocol run.

We compare OptiSwap against the FairSwap protocol implementation of the file sale of 1 GByte with 512 Byte-sized file chunks. In this setting, the circuit depth is $\delta = 21$ and the max fan-in is $\ell = 2$. When we analyze the costs for the OptiSwap protocol we distinguish between deployment costs and the execution costs in the optimistic and pessimistic case (cf. Table 2). We also analyze an optimized version of the contracts which we call split contract. Here we apply the idea from [20], where we create two versions of the judge contract. The first part is responsible for the execution in the optimistic case and the second part only for the dispute. The key idea is that we only need to deploy the second contract in case the parties start the dispute. This reduces the costs for the purely optimistic case. In case of the file sale application the complex logic for the interactive dispute can be encapsulated in the dispute contract which makes the optimistic case even slightly cheaper compared to FairSwap. The authors of [8] claim deployment costs of 1 050 000 gas for the file sale implementation of the FairSwap-protocol. It is reasonable that this value is much less than the deployment costs of the standalone version of OptiSwap, since the challenge-response functionality is not required in FairSwap. However, our optimized split contract implementation shows that the deployment costs for an optimistic execution can be even further reduced. A more extensive analysis and comparison of gas costs is given in the full version of this paper [9].

Optimistic case execution. Without a complaint the protocol runs in either 4 or 5 rounds, depending on the Buyer. In comparison to FairSwap, the first message of OptiSwap does not contain information about $\phi(\mathbf{x})$ and the Seller does not have to compute $\phi(\mathbf{x})$ in this case. This reduces both the message and computation complexity. The biggest part of transaction fees in this case are the deployment fees (cf.

[4]This memory size is based on the integer representation in Solidity.

[5]We consider an average gas price of 14.4 GWei and an exchange rate of 181.57 USD per ether. This data is taken from https://etherscan.io on November 18, 2019.

Table 2) which are paid by the Seller in the first round[6]. The costs for deployment and protocol fees of the Seller can be taken into account when defining the price for the digital good. But recall, that the optimistic case can end in two ways; either the Buyer confirms that he received $\mathbf{x}$ and triggers the payout, or the Seller has to request it after the Buyer did neither confirm nor complain. Fee security for the Seller can be achieved by letting the Buyer lock a small security deposit in the second round. The Seller will receive this deposit on top of the payment unless the Buyer correctly confirms the exchange in round 4; in which case he gets this deposit back.

Pessimistic case execution. On top of the four rounds of the honest execution, the parties execute the interactive dispute procedure. This includes at most a_ϕ challenge-response repetitions. We used the basic challenge-response strategy resulting in $a_\phi = ((\delta - 1) * \ell + 1) = ((21 - 1) * 2 + 1) = 41$. This also means that the costs for the pessimistic case execution grow significantly due to the higher transaction load (cf. Table 2). Contrary, the interactive dispute procedure is not required in FairSwap, since all the data is already transferred in the first protocol message. Therefore, FairSwap requires only one transaction from the Buyer providing the PoM costing at most 194 068 gas. Comparing to FairSwap, the number of rounds and the costs of the dispute resolution can be considered as a downside of OptiSwap. However, we like to stress that an honest party always gets compensated at the end of the protocol. This also implies a deterrent effect on malicious parties.

Within the challenge-response procedure, the Buyer will send a relatively small challenge query but forces the Seller to respond with a potentially large transaction. For our implementation of the file sale application, a challenge transaction costs at most 70 057 gas and a response costs at most 645 331 gas. While the maximum number of rounds is fixed, it will be hard in most cases to precisely predict the costs for the transactions. It would be easy for the Buyer to grieve the Seller – force him (at a low cost) to pay a lot of transaction fees to show his correct behavior. The transaction fees are many times higher than the honest execution and could even outweigh the price p. A rational Seller could be forced this way to waive the payment in order to save transaction fees.

To prevent grieving attacks, we present a simple security fee mechanism that should guarantee that an honest party does not carry the costs for the dispute. In particular, the Seller should be compensated if the Buyer cannot prove misbehavior and the Buyer should be compensated if the Seller cheated. To ensure this, both parties need to add some additional coins to each transaction executed in the challenge-response procedure. All coins will be collected in the contract and paid out together with the payment. This ensures that the honest party will receive its deposit back and gets the malicious party's deposit as compensation.

[6]Without loss of generality the deployment costs could also be carried by the Buyer in the second round, if he gets the auxiliary information from the Seller first.

Since the response transactions of the Seller are more expensive and the Buyer might need to compensate for them, his deposit $f_{\mathcal{B}}$ will be higher than the one of the Seller $f_{\mathcal{S}}$. The parameters depend on the circuit and are contained in the auxiliary information aux. The value must be large enough to compensate for the worst case transaction size. As a result, it might happen that one or both parties deposit much higher fees than the actual costs for the blockchain transactions. However, since the honest party gets his complete money back, this aspect just increases the temporal deposit costs. In addition, it provides a highly deterrent effect for malicious party and disincentives cheating. We give concrete values for $f_{\mathcal{B}}$ and $f_{\mathcal{S}}$ for an exemplary file sale application in the full version of this paper [9].

We note that the Buyer can still grieve the Seller by forcing him to pay the deployment fees in the first round. The Seller pays the deployment fees with the risk that the Buyer aborts and does not send his message in the second round. At this point there is no way to force the Buyer to compensate the Seller for the invested fees.

5 SECURITY ANALYSIS

We formally prove the security of OptiSwap in the Universal Composability (UC) framework (cf. the full version of this paper [9] for an introduction to the UC model and for the full proof). We start by informally arguing about the security properties to give the reader a intuition about the achieved security of the OptiSwap protocol. Then we define the ideal behavior of OptiSwap by the ideal functionality $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$ which captures the overall security of our construction. Afterwards we present a short sketch of our security proof.

5.1 Informal Security Discussion

In this section we will informally argue why OptiSwap achieves all security and efficiency properties. In every step of the OptiSwap protocol the parties only have a limited time to send an expected transaction. On one hand this limits the maximal round time while on the other hand it identifies if a party aborted. If one of the parties aborts, the other party calls a timeout-function in the contract. This triggers the judge to verify that the timeout indeed expired and it will terminate after it sent all remaining coins to the opponent. It is important for the security of honest parties that this timeout is chosen large enough, such that honest transactions will be included in the blockchain in time.

Security against malicious Buyers (S1). This property guarantees to the honest Seller that he will get paid if the Buyer learns the witness x. In particular it means that (a) the Buyer learns nothing about x before k is revealed and (b) he cannot forge a proof of misbehavior if the Seller behaves honestly. (a) holds because of the CPA-secure encryption, which guarantees that the ciphertext $\mathbf{z}$ does not leak information about the plaintext $\mathbf{x}$ and the hiding property of the commitment which hides k from the Buyer. This makes it impossible for the Buyer to learn $\mathbf{x}$ without breaking any of the two schemes. (b) is guaranteed because the honest Seller

will not cheat with the computation of $\phi(\mathbf{x})$ which makes it impossible for the Buyer to claim this (since he cannot find collisions for hash values in the Merkle tree commitment of $\mathbf{z}$). Additionally, E3 ensures that the Buyer cannot delay the payout forever.

Security against malicious Sellers (S2). This property protects the Buyer and his funds. It guarantees that – no matter what the Seller does – the Buyer will either receive the correct witness $\mathbf{x}$ or get his money back. If the Seller aborts or reveals a wrong key the contract will automatically reimburse the Buyer. This follows from the fact that it is cryptographically hard to break the binding of the commitment c. If the Seller reveals the correct key and the witness is correct, property S2 is also satisfied. So the most interesting case for the analysis is the situation when the Buyer received an incorrect witness $\mathbf{x}' \neq \mathbf{x}$ s.t. $\phi(\mathbf{x}') = 0$. In this case the interactive dispute procedure guarantees that the Buyer will be able to find a statement (PoM) which will make the judge refund the payment to him. Unless the Seller aborts, this statement will either prove that $\phi(\mathbf{x}') = 0$, that the Seller lied about the result of some gate $\phi_i \in \phi$ or that he used a witness $\mathbf{x} \neq \mathsf{Dec}(k, \mathbf{z})$. This guarantee follows from the fact that the Buyer has enough rounds in the dispute resolution to challenge the gates required for creating a PoM and that Seller is forced by the judge to respond to all queries.

Security against grieving (S3). This property protects both parties. As seen in Section 4.2, OptiSwap protects against grieving attacks by using transaction deposits for every step in the interactive dispute phase. These deposits will be paid out to the Buyer if he finds a PoM and to the Seller if no PoM can be provided. From properties S1 and S2 we know that the honest party will always receive the coins. This guarantees to honest parties that all paid dispute fees (and more) will be reimbursed after the dispute has finished.

Communication complexity (E1). The communication complexity in the optimistic case of OptiSwap is mainly determined by the first message of the protocol. All following contract transactions have a fixed size that are independent of the witness $\mathbf{x}$ and the circuit ϕ. The first message has a size of $|\mathbf{z}|$ which is identical to $|\mathbf{x}|$ since it is its encryption. Therefore, the overall communication complexity is $\mathcal{O}(\mathbf{x})$ which satisfies property (E1). This is in contrast to FairSwap [8] where the first message depends on both, the witness and the circuit. In the pessimistic case the message complexity is depended on ϕ. A challenge consists of a single integer while a response contains the output value of a gate along with a Merkle tree proof. The size of a gate output value is at most the size of a witness chunk $\frac{|\mathbf{x}|}{n}$. Hence, the size of a response is at most $\frac{|\mathbf{x}|}{n} + (1 + \log(m))$. The number of challenges is limited by a_ϕ resulting in an overall complexity of a_ϕ integers for the Buyer and $a_\phi \times (\frac{|\mathbf{x}|}{n} + (1 + \log(m)))$ values for the Seller in the worst case. Note that in this case (S3) guarantees that any honest party forced to engage in this procedure, will get compensated.

Round complexity (E2+E3). In the optimistic case the protocol runs either 4 or 5 rounds (in case the Buyer doesn't confirm that he received the witness). This satisfies (E2). In the pessimistic case (E3) the challenge-response procedure is started in the fourth round, and will take at most a_ϕ repetitions and finish with an additional PoM transaction of the Buyer. In case the Buyer does not send this last message, the Seller has to request his payment in the next round, leading to $5 + 2a_\phi$ rounds in the worst case.

5.2 Ideal Functionality $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$

The functionality $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$ represents a fair exchange of digital goods within a blockchain-based setting. It describes an exchange of a digital good $\mathbf{x}$ between a Seller $\mathcal{S}$ and a Buyer $\mathcal{B}$. While Seller offers the digital good, Buyer must pay for it. The correctness of the witness $\mathbf{x}$ is defined by a circuit ϕ which either outputs 1 for the correct witness or 0 otherwise. It extends the functionality $\mathcal{F}_{\mathsf{cfe}}^{\mathcal{L}}$ for coin aided fair exchange [8] and also utilizes an idealized ledger functionality $\mathcal{L}$ for the on-chain handling of coins. We do not specify the ledger ideal functionality $\mathcal{L}$ here but refer the reader to the full version of this paper [9]. The major change in comparison to $\mathcal{F}_{\mathsf{cfe}}^{\mathcal{L}}$ is the modeling of an interactive challenge-response procedure in case of dispute. This includes modeling security fees incorporated in our construction.

Ideal Functionality $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$

The ideal functionality $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$ (in session id) for interactive coin aided fair exchange interacts with Seller $\mathcal{S}$ and Buyer $\mathcal{B}$. Moreover, it has access to the global ledger functionality $\mathcal{L}$ and interacts with the ideal adversary Sim.

Initialization

(Round 1) Upon receiving $(sell, id, \mathbf{x}, \phi, p, f_\mathcal{S}, f_\mathcal{B})$ with $p, f_\mathcal{S}, f_\mathcal{B} \in \mathbb{N}$ and $\phi(\mathbf{x}) = 1$ from $\mathcal{S}$, store witness $\mathbf{x}$, circuit ϕ with challenge limit property a_ϕ, price p, and fee parameters $f_\mathcal{S}, f_\mathcal{B}$ and leak $(sell, id, \phi, p, f_\mathcal{S}, f_\mathcal{B}, \mathcal{S})$ to Sim.

Upon receiving $(sell\text{-}fake, id, \mathbf{x}, \phi, p, f_\mathcal{S}, f_\mathcal{B})$ with $p, f_\mathcal{S}, f_\mathcal{B} \in \mathbb{N}$ from corrupted Seller $\mathcal{S}^*$, store witness $\mathbf{x}$, circuit ϕ with challenge limit property a_ϕ, price p, and fee parameters $f_\mathcal{S}, f_\mathcal{B}$.

(Round 2) Upon receiving $(abort, id)$ from Seller $\mathcal{S}$, leak $(abort, id, \mathcal{S})$ to Sim, send $(aborted, id)$ to $\mathcal{B}$, and terminate.

Upon receiving (buy, id, ϕ) from Buyer $\mathcal{B}$, send $(freeze, id, \mathcal{B}, p)$ to $\mathcal{L}$. If $\mathcal{L}$ responds with $(frozen, id, \mathcal{B}, p)$, leak $(buy, id, \mathcal{B})$ to Sim and go to *Revealing* phase.

If no message is received during round 2, terminate.

Revealing

(Round 3) Upon receiving $(abort, id)$ from Buyer $\mathcal{B}$, leak $(abort, id, \mathcal{B})$ to Sim, send $(unfreeze, id, \mathcal{B})$ to $\mathcal{L}$, $(aborted, id)$ to $\mathcal{S}$, and terminate.
Upon receiving $(abort, id)$ from corrupted Seller $\mathcal{S}^*$ in round 3, send $(unfreeze, id, \mathcal{B})$ to $\mathcal{L}$ in the next round and terminate.
If no message is received in round 3, send $(revealed, id, \mathbf{x})$ to $\mathcal{B}$, set $s = challenge$, wait one round, and go to *Interaction* phase.

Interaction

Upon receiving $(freeze, id, \mathcal{B}, a_r)$ with $a_\phi \geq a_r$ from Sim when $s = challenge$, send $(freeze, id, \mathcal{B}, a_r * f_\mathcal{B})$ to $\mathcal{L}$. If $\mathcal{L}$ responds with $(frozen, id, \mathcal{B}, a_r * f_\mathcal{B})$, store a_r, set $a_\phi = a_\phi - a_r$, $s = response$, and wait one round.
Upon receiving $(freeze, id, \mathcal{S}, a_s)$ with $a_s = a_r$ from Sim when $s = response$, send $(freeze, id, \mathcal{S}, a_s * f_\mathcal{S})$ to $\mathcal{L}$. If $\mathcal{L}$ responds with $(frozen, id, \mathcal{S}, a_s * f_\mathcal{S})$, set $s = challenge$ and wait one round.
Upon receiving $(abort, id, \Delta)$ from corrupted Buyer $\mathcal{B}^*$, where $\Delta \in \{0, 1\}$, when $s = challenge$, wait Δ rounds. Then send $(unfreeze, id, \mathcal{S})$ to $\mathcal{L}$ and $(sold, id)$ to $\mathcal{S}$ and terminate.
Upon receiving $(abort, id, \Delta)$ from corrupted Seller $\mathcal{S}^*$, where $\Delta \in \{0, 1\}$, when $s = response$, wait Δ rounds. Then send $(unfreeze, id, \mathcal{B})$ to $\mathcal{L}$ and $(not\ bought, id, \mathbf{x})$ to $\mathcal{B}$ and terminate.
If no message is received, execute *Payout* phase.

Payout

If $\phi(\mathbf{x}) = 1$, send messages $(unfreeze, id, \mathcal{S})$ to $\mathcal{L}$, $(sold, id)$ to $\mathcal{S}$, and $(bought, id, \mathbf{x})$ to $\mathcal{B}$. Otherwise, if $\phi(\mathbf{x}) \neq 1$, send messages $(unfreeze, id, \mathcal{B})$ to $\mathcal{L}$, $(not\ sold, id)$ to $\mathcal{S}$, and $(not\ bought, id, \mathbf{x})$ to $\mathcal{B}$. Terminate the execution after the payout.

We describe the ideal functionality in the following. We first assume two honest parties and describe additional opportunities for malicious parties afterwards.
The ideal functionality $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$ starts in the *initialization* phase during which both parties provide their initial input. In the first round, Seller starts the execution by sending the witness $\mathbf{x}$, the verification circuit ϕ, a price p, and fee parameters $f_\mathcal{S}, f_\mathcal{B}$ to $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$. For an honest Seller it must hold that $\phi(\mathbf{x}) = 1$ in the initialization. In the second round, Seller may abort the execution or he waits for Buyer to accept the offer. By accepting the offer, the ideal functionality instructs the ledger functionality to lock p coins from Buyer $\mathcal{B}$. If locking p coins from $\mathcal{B}$ is not successful due to insufficient funds, the functionality ends the interactive fair exchange protocol.

In the *revealing* phase in round 3, Buyer may abort the protocol execution and receives his coins back or he waits to learn the witness $\mathbf{x}$. After the witness is revealed, the *interaction* phase is started. Assuming both parties are honest, Seller sent the correct witness and Buyer has no intention to challenge Seller. Therefore, the *payout* phase is executed during which Seller gets the money and the fair exchange is completed.

Next, considering the case that Seller is malicious. He can send an incorrect witness $\mathbf{x}$ such that $\phi(\mathbf{x}) \neq 1$ to $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$ in the *initialization* phase. Since a malicious Seller has not to follow the protocol, he may abort in round 3 by not sending the required message. If he aborts in round 3 before the witness is revealed to Buyer $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$ instructs the ledger functionality $\mathcal{L}$ to unfreeze the locked money in Buyer's favor. In case Seller does not abort, $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$ reveals the incorrect witness to honest Buyer. Since an honest Buyer is not willing to pay for an incorrect witness, he starts the dispute resolution sub-protocol by challenging Seller. This challenge-response interaction is simulated by the ideal adversary Sim within the *interaction* phase. Each challenge or response message costs some fees which are locked by $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$ by instructing $\mathcal{L}$ to freeze these fees. At each round during the *interaction* phase in which Seller must provide a response to a challenge query a malicious Seller may abort. In this case, Buyer gets all the money that is locked by the ledger functionality. After several interactions, which is limited by the challenge limit property a_ϕ of the circuit ϕ, Buyer is able to generate a valid proof of misbehavior within the protocol. The ideal functionality executes the *payout* phase and sends all the money to $\mathcal{B}$, since malicious Seller offered an incorrect witness at the start of the execution.

In addition to the already explained actions, a malicious Buyer may challenge Seller even tough he provided the correct witness during the *initialization* phase. These challenges are simulated by the ideal adversary in the *interaction* phase. Again, the number of possible challenges is limited by the challenge limit property a_ϕ of the circuit ϕ. As explained above, a malicious Seller may abort within the *interaction* phase and a malicious Buyer may do this as well. In this case, all the locked money gets unfrozen by the ledger functionality in favor of the Seller. If Seller has sent the correct witness in the *initialization* phase, a malicious Buyer will not be able to generate a valid proof of misbehavior and Seller is able to respond to every challenge posed by Buyer. This ends in the execution of the *payout* phase during which the ideal functionality $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$ instructs $\mathcal{L}$ to unfreeze all the money in favor of $\mathcal{S}$.

In order to consider delayed messages during a protocol execution, the ideal adversary Sim is able to delay the execution of $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$. Whenever Sim obtains the instruction from the environment to delay the message of any corrupted party by δ rounds, the simulator instructs the ideal functionality

to delay the execution by δ rounds. For sake of clarity, this description is omitted in the definition.

5.3 Proof Sketch

In this version of the paper we only provide a sketch of the security proof and refer the reader to the full version [9] for a formal proof. We follow the model of FairSwap [8] and assume synchronous communication and static corruption. As we work in the generalized universal composability (GUC) framework [7] (cf. full version for definition of ledger $\mathcal{L}$ and random oracle $\mathcal{H}$) the formal security statement is as follows:

THEOREM 5.1. *The* OPTISWAP *protocol* Π *stated in Section 3.4 GUC-realizes the ideal functionality* $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$ *within the judge smart contract* $(\mathcal{G}_{\mathsf{jc}}^{\mathcal{L},\mathcal{H}}, \mathcal{L}, \mathcal{H})$*-hybrid world, where* $\mathcal{L}$ *denotes a ledger functionality and* $\mathcal{H}$ *is modeled as a global random programmable oracle.*

To prove this theorem, we need to show that for any possible corruption case, the environment cannot distinguish the ideal world execution from the execution of the OPTISWAP protocol. In particular, the outputs of honest parties and all changes in the global functionalities ($\mathcal{L}$, $\mathcal{H}$) must be identical. This must also hold for all messages sent to the corrupted parties. We prove this by constructing a simulator *Sim* which simulates the execution of OPTISWAP in the real world, by interacting with the ideal world functionality and corrupted parties in the ideal world. In the following we highlight the role of the simulator in all four corruption cases.

Two honest parties. In case of two honest parties, we need to assume a secure channel between the Seller and the Buyer that is used for transferring the first protocol message. In this secure channel, the adversary only learns that information is transferred but he gains no knowledge about the content of the message. Without such a secure channel, the simulator would have to create an encrypted witness $\mathbf{z}^*$ without ever learning the correct witness $\mathbf{x}$.

Since both parties are honest, the execution of the protocol only starts if the witness is correct, i.e., $\phi(\mathbf{x}) = 1$. This is ensured by the honest Seller in the hybrid world execution and by the ideal functionality $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$ in the ideal world execution. Therefore, the environment obtains the correct witness $\mathbf{x}$ back after the execution in both worlds and cannot use the input in order to distinguish between the two executions. The commitments created by the honest Seller in the first protocol message depend on a randomly chosen encryption key. Based on the randomly selected keys and the hiding property of the commitment scheme, the environment cannot distinguish between the values computed by the honest Seller and the one created by the simulator.

Finally, a simulation of abort is possible in a straightforward way and the simulator ensures that money is locked and unlocked in the same round as in the real world execution.

Corrupted Seller. For the proof of Theorem 5.1 in case of a corrupted Seller, we make use of the hybrid argument technique and show indistinguishability by a sequence of security games. The main challenge in this corruption scenario is that the simulator has to provide input to $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$ on behalf of the corrupted Seller without knowing the witness $\mathbf{x}$. The simulator only learns the encrypted witness $\mathbf{z}$ and the commitment to the key c in the first round. We use the observability feature of the programmable random oracle $\mathcal{H}$ at this point. We need to distinguish two cases of how the environment creates the commitment c:

- c is constructed correctly by querying $\mathcal{H}$: in this case, we use the observability feature of $\mathcal{H}$ to obtain the key k. This allows the simulator to decrypt $\mathbf{z}$ and provide the witness $\mathbf{x}$ as input to $\mathcal{F}_{\mathsf{icfe}}^{\mathcal{L}}$. Only if the environment finds a collision of $\mathcal{H}$, he can distinguish between the two executions. However, based on the binding property of the commitment scheme, this happens only with negligible probability.
- c is constructed incorrectly: in this case $\mathcal{H}$ was not queried before, so the simulator cannot take advantage of the observability feature. However, the environment has to provide correct opening values k and d in round 3 of the protocol, which he can only guess with probability $\frac{1}{2^\mu}$. This is negligible for large μ.

Finally, the simulator has to simulate the interactive dispute resolution procedure.

Corrupted Buyer. The main challenge of the simulation in case of a corrupted Buyer is that the simulator needs to create an encrypted witness $\mathbf{z}^*$ in the first round and to present an encryption key in the third round such that the decryption of $\mathbf{z}^*$ equals the correct witness $\mathbf{x}$ which is known to the simulator only after round 3. To achieve indistinguishability, we make use of the programming feature of $\mathcal{H}$. The encrypted witness $\mathbf{z}^*$ is chosen randomly by the simulator. Based on the IND-CPA-security of the encryption scheme, the environment cannot distinguish between the encrypted witness $\mathbf{z}$ created by the honest Seller and $\mathbf{z}^*$. After the correct witness $\mathbf{x}$ is revealed in the third round, the simulator programs $\mathcal{H}$ such that the decryption of the $\mathbf{z}^*$ equals $\mathbf{x}$.

Although the honest Seller provides a correct witness $\mathbf{x}$, a corrupted Buyer may start the dispute resolution procedure. Therefore, the simulator needs to simulate the interaction. He keeps track of the challenge limit parameter to ensure that the corrupted Buyer may not pose more challenges than allowed. Moreover, the honest Seller is always able to create valid responses and the simulator has enough information to simulate the same behavior.

Two corrupted parties. In case of two corrupted parties, the OPTISWAP protocol does not provide any guarantees. In particular, termination is not given and coins may be locked forever. However, the simulation in this corruption scenario need to be shown in order to allow composability.

The simulation is a combination of the single corruption cases and in most aspects straightforward. One difference is the possibility that money is blocked in the judge smart contract. To achieve the same behavior in the simulation, we make use of the blocking feature of the ledger functionality

$\mathcal{L}$. This feature allows the simulator to block an unfreezing of coin on behalf of a corrupted party. The result is that the money is locked forever.

6 CONCLUSION AND FUTURE WORK

We presented OptiSwap, a smart contract based two-party protocol for realizing a fair exchange for digital commodity against money. In comparison to already existing fair exchange protocols, we significantly improved the execution of the optimistic case in which both parties behave honestly. We integrated an interactive dispute handling in OptiSwap that is only run in the pessimistic case. This allows us to have almost no overhead in communication complexity in the optimistic case.

Furthermore, OptiSwap contains a protection mechanism against so-called grieving attacks, where the attacker tries to harm the honest party by forcing him to pay fees. The protection is based on transaction fees paid by both parties such that the honest party is reimbursed at the end of the protocol execution. We provide a reference implementation of our judge smart contract for Ethereum. Based on that, we estimated gas costs for the optimistic and pessimistic case.

Future work. The execution costs in the optimistic mode strongly depend on the deployment costs of the judge smart contract. By allowing repeated exchanges over one contract, the costs for a single execution might be significantly reduced. An interesting research question is the analysis of minimal bounds on communication and computation complexity. Results can show how far further our construction might be improved.

7 ACKNOWLEDGMENTS

This work was partly funded by the iBlockchain project (grant nr. 16KIS0902) funded by the German Federal Ministry of Education and Research (BMBF), by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – SFB 1119 – 236615297, and by the German Federal Ministry of Education and Research and the Hessen State Ministry for Higher Education, Research and the Arts within their joint support of the National Research Center for Applied Cybersecurity ATHENE.

REFERENCES

[1] Marcin Andrychowicz, Stefan Dziembowski, Daniel Malinowski, and Lukasz Mazurek. 2014. Secure Multiparty Computations on Bitcoin. In *2014 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 443–458. https://doi.org/10.1109/SP.2014.35

[2] N. Asokan, Victor Shoup, and Michael Waidner. 1998. Optimistic Fair Exchange of Digital Signatures (Extended Abstract). In *EUROCRYPT'98 (LNCS)*, Kaisa Nyberg (Ed.), Vol. 1403. Springer, Heidelberg, 591–606. https://doi.org/10.1007/BFb0054156

[3] Iddo Bentov and Ranjit Kumaresan. 2014. How to Use Bitcoin to Design Fair Protocols. In *CRYPTO 2014, Part II (LNCS)*, Juan A. Garay and Rosario Gennaro (Eds.), Vol. 8617. Springer, Heidelberg, 421–439. https://doi.org/10.1007/978-3-662-44381-1_24

[4] Bitcoin Wiki. [n.d.]. Zero Knowledge Contingent Payment. https://en.bitcoin.it/wiki/. (Accessed on 09/19/2019).

[5] Christian Cachin and Jan Camenisch. 2000. Optimistic Fair Secure Computation. In *CRYPTO 2000 (LNCS)*, Mihir Bellare (Ed.), Vol. 1880. Springer, Heidelberg, 93–111. https://doi.org/10.1007/3-540-44598-6_6

[6] Matteo Campanelli, Rosario Gennaro, Steven Goldfeder, and Luca Nizzardo. 2017. Zero-Knowledge Contingent Payments Revisited: Attacks and Payments for Services. In *ACM CCS 17*. ACM Press, 229–243. https://doi.org/10.1145/3133956.3134060

[7] Ran Canetti, Yevgeniy Dodis, Rafael Pass, and Shabsi Walfish. 2007. Universally Composable Security with Global Setup. In *TCC 2007 (LNCS)*, Salil P. Vadhan (Ed.), Vol. 4392. Springer, Heidelberg, 61–85. https://doi.org/10.1007/978-3-540-70936-7_4

[8] Stefan Dziembowski, Lisa Eckey, and Sebastian Faust. 2018. FairSwap: How To Fairly Exchange Digital Goods. In *ACM CCS 18*. ACM Press, 967–984. https://doi.org/10.1145/3243734.3243857

[9] Lisa Eckey, Sebastian Faust, and Benjamin Schlosser. 2019. OptiSwap: Fast Optimistic Fair Exchange. *IACR Cryptology ePrint Archive* 2019 (2019), 1330. https://eprint.iacr.org/2019/1330

[10] Oded Goldreich. 2004. *Foundations of Cryptography: Basic Applications*. Vol. 2. Cambridge University Press, Cambridge, UK.

[11] Oded Goldreich, Silvio Micali, and Avi Wigderson. 1987. How to Play any Mental Game or A Completeness Theorem for Protocols with Honest Majority. In *19th ACM STOC*, Alfred Aho (Ed.). ACM Press, 218–229. https://doi.org/10.1145/28395.28420

[12] Mathias Hall-Andersen. 2019. FastSwap: Concretely Efficient Contingent Payments for Complex Predicates. *IACR Cryptology ePrint Archive* 2019 (2019), 1296. https://eprint.iacr.org/2019/1296

[13] Harry A. Kalodner, Steven Goldfeder, Xiaoqi Chen, S. Matthew Weinberg, and Edward W. Felten. 2018. Arbitrum: Scalable, private smart contracts. In *27th USENIX Security Symposium, USENIX Security 2018, Baltimore, MD, USA, August 15-17, 2018*, William Enck and Adrienne Porter Felt (Eds.). USENIX Association, 1353–1370. https://www.usenix.org/conference/usenixsecurity18/presentation/kalodner

[14] Aggelos Kiayias, Hong-Sheng Zhou, and Vassilis Zikas. 2016. Fair and Robust Multi-party Computation Using a Global Transaction Ledger. In *EUROCRYPT 2016, Part II (LNCS)*, Marc Fischlin and Jean-Sébastien Coron (Eds.), Vol. 9666. Springer, Heidelberg, 705–734. https://doi.org/10.1007/978-3-662-49896-5_25

[15] Ranjit Kumaresan and Iddo Bentov. 2016. Amortizing Secure Computation with Penalties. In *ACM CCS 16*, Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi (Eds.). ACM Press, 418–429. https://doi.org/10.1145/2976749.2978424

[16] Ranjit Kumaresan, Vinod Vaikuntanathan, and Prashant Nalini Vasudevan. 2016. Improvements to Secure Computation with Penalties. In *ACM CCS 16*, Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi (Eds.). ACM Press, 406–417. https://doi.org/10.1145/2976749.2978421

[17] Alptekin Küpçü and Anna Lysyanskaya. 2010. Usable Optimistic Fair Exchange. In *CT-RSA 2010 (LNCS)*, Josef Pieprzyk (Ed.), Vol. 5985. Springer, Heidelberg, 252–267. https://doi.org/10.1007/978-3-642-11925-5_18

[18] Henning Pagnia and Felix C Gärtner. 1999. *On the impossibility of fair exchange without a trusted third party*. Technical Report. Technical Report TUD-BS-1999-02, Darmstadt University of Technology.

[19] Jason Teutsch and Christian Reitwießner. 2019. A scalable verification solution for blockchains. *CoRR* abs/1908.04756 (2019). arXiv:1908.04756 http://arxiv.org/abs/1908.04756

[20] Eric Wagner, Achim Völker, Frederik Fuhrmann, Roman Matzutt, and Klaus Wehrle. 2019. Dispute Resolution for Smart Contract-based Two-Party Protocols. In *IEEE International Conference on Blockchain and Cryptocurrency, ICBC 2019, Seoul, Korea (South), May 14-17, 2019*. IEEE, 422–430. https://doi.org/10.1109/BLOC.2019.8751312

[21] Andrew Chi-Chih Yao. 1986. How to Generate and Exchange Secrets (Extended Abstract). In *27th FOCS*. IEEE Computer Society Press, 162–167. https://doi.org/10.1109/SFCS.1986.25